ROS 2 - Sistema Operativo de Robótica

Proyecto de investigación - Sistemas de Comunicaciones

Pablo Sebastian Gomez · Irina Terebiznik · Matias Peña

Sistemas de Comunicaciones



Universidad Tecnológica Nacional
Facultad Regional Avellaneda
Departamento de Ingeniería Electrónica

Índice

1. Capitulo 1 **3**

1.1. Introducción del trabajo . . . 3
1.2. Objetivo y alcance . . . 3
1.3. Metodología y Fuentes de investigación . . . 4
1.4. Delimitación del estudio . . . 4
1.5. Universo Espacio Temporal . . . 5

2. Capitulo 2 **5**

2.1. Introducción del tema . . . 5
2.2. Descripción del contexto del tema . . . 6
2.3. Mercado y Segmentación del tema . . . 7
2.4. Legislación del tema . . . 7

3. Capitulo 3 **7**

3.1. Fundamentos técnicos . . . 7
3.2. Ámbitos de aplicación . . . 7
3.3. Estrategias Competitivas . . . 7
3.4. Normativas asociadas . . . 7

4. Capitulo 4 **8**

4.1. Conclusiones . . . 8
4.2. Glosario de términos . . . 8
4.3. Referencias . . . 8

Extracto

El presente informe tiene como objetivo presentar un análisis detallado del Sistema Operativo de Robótica (ROS 2), explorando sus características, arquitectura y aplicaciones en el ámbito de la robótica. Se abordarán aspectos clave como la comunicación entre nodos, la gestión de sensores y actuadores, y la integración con hardware específico. Además, se discutirán las ventajas y desafíos asociados con el uso de ROS 2 en proyectos de investigación y desarrollo.

1. Capitulo 1

1.1. Introducción del trabajo

El presente trabajo de investigación se desarrolla en el marco de la asignatura Sistemas de Comunicaciones de la Universidad Tecnológica Nacional. Su propósito es analizar ROS (*Robot Operating System*) desde la perspectiva de los sistemas de comunicaciones, considerando los mecanismos de intercambio de información que emplea y los estándares sobre los que se sustenta.

Los sistemas robóticos actuales constituyen, en esencia, sistemas distribuidos. Sensores, actuadores, unidades de procesamiento y estaciones de supervisión operan como procesos independientes, frecuentemente alojados en distintos equipos, que deben intercambiar datos de manera continua a través de una red. En este contexto, la arquitectura de comunicaciones deja de ser un aspecto secundario y pasa a condicionar directamente la latencia, la confiabilidad y la escalabilidad del sistema en su conjunto.

ROS se ha consolidado como el *framework* de referencia para el desarrollo de software robótico, tanto en el ámbito académico como en el industrial. Su evolución resulta particularmente relevante para el estudio de las comunicaciones: la primera versión (ROS 1) se basa en una arquitectura híbrida que depende de un nodo maestro central, mientras que ROS 2 adopta un esquema descentralizado sustentado en el estándar DDS (*Data Distribution Service*), el protocolo RTPS (*Real-Time Publish-Subscribe*) y el transporte sobre UDP/IP. Este trabajo aborda dicha transición, analizando sus fundamentos técnicos, la configuración de la calidad de servicio (QoS) y las implementaciones de *middleware* disponibles.

1.2. Objetivo y alcance

El objetivo general del trabajo es analizar el estado del arte de ROS, con foco en la evolución de su arquitectura de comunicaciones desde ROS 1 hasta ROS 2 y en los estándares y protocolos que la sustentan.

Para ello se plantean los siguientes objetivos específicos:

- Describir los conceptos arquitectónicos de ROS (nodos y *topics*) y sus patrones de comunicación (*services* y *actions*).
- Analizar la arquitectura híbrida de ROS 1 y las limitaciones derivadas de su dependencia del nodo maestro (*roscore*) como punto único de falla.

- Describir el modelo descentralizado de ROS 2 y la pila de comunicaciones DDS, RTPS y UDP/IP.
- Analizar los mecanismos de calidad de servicio (QoS) y su configuración.
- Relevar el soporte de ROS 2 para sistemas operativos de tiempo real (RTOS) y microcontroladores mediante micro-ROS.
- Describir la capa de abstracción de *middleware* (RMW) y comparar sus principales implementaciones: Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS.
- Presentar el esquema de distribuciones de ROS y su vinculación con las versiones de Ubuntu.
- Realizar una demostración práctica sobre hardware dedicado (Raspberry Pi) que ilustre el intercambio de mensajes entre nodos de ROS 2 a través de la red.

En cuanto al alcance, el estudio se centra en la capa de *middleware* y transporte de datos, complementado con una prueba de concepto de carácter demostrativo. No se abordan en profundidad los algoritmos de percepción, planificación o control, ni el desarrollo de aplicaciones robóticas específicas.

1.3. Metodología y Fuentes de investigación

La investigación es de tipo descriptivo y de enfoque cualitativo. Se basa en el análisis documental de las fuentes técnicas que definen la arquitectura de ROS y de los estándares de comunicaciones sobre los que se apoya. Los resultados del análisis se contrastan con una prueba de concepto sobre hardware dedicado.

Las fuentes consultadas se agrupan en las siguientes categorías:

- **Documentación oficial de ROS:** documentación de ROS~2 y propuestas de mejora (REP, *ROS Enhancement Proposals*), en particular la REP~2000, que define las plataformas soportadas por cada distribución.
- **Especificaciones de estándares:** especificaciones de la OMG (*Object Management Group*) para DDS y DDSI-RTPS, y documentos RFC de la IETF para los protocolos UDP (RFC~768) e IP (RFC~791).
- **Documentación de fabricantes:** documentación técnica de las implementaciones Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS, y del proyecto micro-ROS.
- **Publicaciones académicas:** artículos científicos sobre el diseño, la arquitectura y el desempeño de ROS 2 y DDS.

1.4. Delimitación del estudio

El análisis se delimita según los siguientes criterios:

- **Sectores:** telecomunicaciones, robótica y dispositivos IoT (*Internet of Things*).
- **Objeto de estudio:** el sistema de comunicaciones de ROS 1 y ROS 2, sin abarcar el resto de las herramientas del ecosistema.
- **Capas analizadas:** *middleware* (RMW y DDS), protocolo de interoperabilidad (RTPS) y transporte (UDP/IP).
- **Plataformas:** distribuciones de ROS sobre Ubuntu Linux, RTOS y microcontroladores a través de micro-ROS.

1.5. Universo Espacio Temporal

En el plano temporal, el estudio abarca la evolución de ROS desde el inicio de su desarrollo en 2007 hasta las distribuciones vigentes de ROS 2 en 2026. Se pone especial énfasis en el período 2021–2026, en el que ROS 2 se consolida como la plataforma de referencia y ROS 1 alcanza el fin de su soporte con la distribución Noetic en 2025.

En el plano espacial, el análisis tiene alcance global, dado que ROS y los estándares asociados son desarrollados y adoptados a nivel internacional. Se incorporan referencias al contexto argentino cuando resultan pertinentes.

2. Capitulo 2

2.1. Introducción del tema

ROS (*Robot Operating System*) es un *framework* de código abierto para el desarrollo de software robótico. A pesar de su nombre, no constituye un sistema operativo en sentido estricto: se ejecuta sobre un sistema operativo anfitrión, principalmente Linux, y provee una capa de *middleware* junto con un conjunto de bibliotecas y herramientas. Su función central es permitir que los distintos componentes de un robot se desarrollen como procesos independientes y se comuniquen entre sí mediante interfaces estandarizadas.

El proyecto se originó en 2007 en el Laboratorio de Inteligencia Artificial de la Universidad de Stanford y continuó su desarrollo en Willow Garage. Actualmente es mantenido por la Open Source Robotics Foundation (OSRF) junto a una amplia comunidad internacional.

Las principales razones de su adopción son:

- **Modularidad:** cada función del robot se implementa como un componente independiente, lo que facilita el desarrollo en paralelo y el mantenimiento.
- **Reutilización:** existe un amplio repositorio de paquetes para navegación, percepción, manipulación y controladores de hardware.
- **Abstracción de hardware:** los componentes se comunican a través de interfaces definidas, independientemente del fabricante del sensor o actuador.

- **Herramientas:** incluye utilidades de visualización, registro, simulación y depuración.
- **Licenciamiento abierto:** se distribuye bajo licencias permisivas (BSD y Apache 2.0), lo que habilita su uso académico e industrial.

ROS 1 fue concebido como una plataforma de investigación, orientada a un único robot operando en una red local confiable. A medida que su uso se extendió a entornos industriales, sistemas multi-robot y redes inalámbricas, sus limitaciones de diseño se hicieron evidentes. En respuesta, se desarrolló ROS 2, rediseñado desde su base sobre estándares industriales de comunicaciones.

La diferencia fundamental entre ambas versiones reside en su arquitectura de comunicaciones. ROS 1 emplea un esquema híbrido en el que un nodo maestro central (*roscore*) gestiona el descubrimiento de los participantes, mientras que el intercambio de datos se realiza mediante protocolos propios (TCPROS y UDPROS). Este diseño introduce un punto único de falla y no ofrece mecanismos para ajustar el comportamiento de la comunicación según las condiciones de la red.

ROS 2, en cambio, adopta una arquitectura descentralizada basada en el estándar DDS (*Data Distribution Service*) y su protocolo de interoperabilidad RTPS. Esta elección elimina la dependencia de un nodo central y habilita la configuración de políticas de calidad de servicio (QoS), el soporte para sistemas operativos de tiempo real y la integración de microcontroladores mediante micro-ROS. Asimismo, incorpora mecanismos de seguridad basados en la especificación DDS-Security, ausentes en ROS 1.

Con la finalización del soporte de ROS 1 en 2025, ROS 2 se ha consolidado como la plataforma vigente para el desarrollo de nuevos sistemas robóticos.

Los aspectos técnicos de cada arquitectura se desarrollan en detalle en el Capítulo 3.

2.2. Descripción del contexto del tema

ROS se utiliza como capa de integración en sistemas robóticos de diversa complejidad, desde plataformas educativas y de investigación hasta vehículos autónomos y robots industriales. En estos sistemas, los componentes de software suelen distribuirse entre varios equipos: computadoras embarcadas en el robot, microcontroladores asociados a sensores y actuadores, y estaciones de supervisión remotas. ROS actúa como el *middleware* que permite que todos estos componentes intercambien información de forma transparente, sin importar en qué equipo se ejecuten.

El software de ROS se publica en **distribuciones**: conjuntos versionados de paquetes que se prueban y se mantienen en conjunto. Desde 2017, ROS 2 lanza una nueva distribución cada año, el 23 de mayo. Las distribuciones de los años pares son de soporte extendido (LTS, *Long Term Support*) y reciben actualizaciones durante cinco años, mientras que las de los años impares tienen un ciclo de soporte de aproximadamente un año y medio. En paralelo existe Rolling, una distribución de desarrollo continuo en la que se incorporan los cambios antes de su inclusión en una versión estable.

Cada distribución está vinculada a una versión LTS específica de Ubuntu, que constituye su plataforma principal de soporte (*Tier 1*), según lo establecido en la REP 2000. Este vínculo se debe

a que los paquetes binarios de ROS se compilan y se prueban contra las bibliotecas del sistema de esa versión de Ubuntu. En consecuencia, la elección de la distribución de ROS determina la versión del sistema operativo a utilizar. Las distribuciones LTS más recientes y su correspondencia son:

- **ROS 1 Noetic Ninjemys** (2020): Ubuntu 20.04. Última distribución de ROS 1, con fin de soporte en mayo de 2025.
- **ROS 2 Humble Hawksbill** (2022): Ubuntu 22.04.
- **ROS 2 Jazzy Jalisco** (2024): Ubuntu 24.04.
- **ROS 2 Lyrical Luth** (2026): Ubuntu 26.04.

Además de Ubuntu, las distribuciones de ROS 2 ofrecen distintos niveles de soporte para otras plataformas, como Windows y Red Hat Enterprise Linux.

2.3. Mercado y Segmentación del tema

Como es el mercado de usabilidad y la segmentación del mismo. Como se usa en el mundo y argentina

2.4. Legislación del tema

Si existen leyes o normas sobre el tema, el contexto mundial o de argentina

3. Capitulo 3

3.1. Fundamentos técnicos

Fundamentos técnicos del trabajo, desarrollos teóricos, información de arquitectura, esquemas, etc.

3.2. Ámbitos de aplicación

Donde se utiliza y como, en que mercados, etc.

3.3. Estrategias Competitivas

¿Competencia? ¿Producto único?

3.4. Normativas asociadas

¿Qué normas tiene asociadas ITU, IEEE?

4. Capitulo 4

4.1. Conclusiones

Al finalizar el ...

4.2. Glosario de términos

Terminología utilizada en el informe:

- **ROS 2:** Robot Operating System 2, un sistema operativo de código abierto para la robótica.
- **Nodo:** Una unidad de ejecución en ROS 2 que realiza tareas específicas y se comunica con otros nodos.

4.3. Referencias

[1] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7ma ed. Oxford University Press, 2015.

[2] Texas Instruments, *LM317 3-Terminal Adjustable Regulator*, Datasheet SLVS044X, 2020. [Disponible en carpeta `references/`].